

1 The Runner-Up Relay Route

2

Every spring, the trail-running guild of Fennwick charts the great relay between the clubhouse at station 1 and the summit lodge at station n . The n stations are joined by m two-way trails, trail i linking stations u_i and v_i with a running time of w_i seconds, and the trail map keeps every station reachable from every other.

A relay route starts at station 1, ends at station n , and may pass through any stations and trails along the way — runners are even allowed to double back and reuse a trail or revisit a station if they wish. The total time of a route is the sum of the running times of every trail crossing made, counted once per crossing.

The guild's champion already holds the record for the fastest possible total time. To keep the rivalry alive, the vice-champion wants to chase a different mark: the smallest total time that is strictly greater than the fastest possible time.

Given the trail map, find that runner-up time — the second-smallest distinct total time over all possible relay routes from station 1 to station n .

Function Description

Implement the function `findSecondShortestTime`.

Parameters

- n : the number of stations.
- m : the number of trails.

```
1 > #include <bits/stdc++.h>
2 long long findSecondShortestTime (int n, int m,
3   vector<vector<int> > arr) {
4   // Write your code here
5 }
6
7 > |
```

Autocomplete

Ln 1, Col 1

Tab: 4 Spaces

Visual S

Test cases Quick test Results

Test case 1

Sample Input 1

```
4
4
1 2 3
2 4 4
1 3 2
3 4 6
```

Sample Output 1

8

Copy to custom in

< Previous

1/2

Next >

59:11

Run

Submit

Section: Coding Programming

50.00

C++17 (g++ 10.3.0)

arr: 2D with m number of rows and 3 columns:
 u: $u[i]$ is one endpoint of trail i .
 v: $v[i]$ is the other endpoint of trail i .
 w: $w[i]$ is the running time of trail i in seconds.

Returns the second-smallest distinct total time of a route from station 1 to station n .

Input Format

The first line contains a single integer n , the number of stations.
 The second line contains a single integer m , the number of trails.
 Each of the next m lines contains three integers u_i, v_i, w_i describing a trail.

Output Format

Print a single integer — the second-smallest distinct total route time in seconds.

Constraints

- $3 \leq n \leq 100000$
- $n-1 \leq m \leq 200000$
- $1 \leq u_i, v_i \leq n, u_i \neq v_i$
- $2 \leq w_i \leq 1000000000$
- The trail network is connected; multiple trails may join the same pair of stations.
- Routes may revisit stations and reuse trails; a runner-up time always exists.

```
1 > #include<bits/stdc++.h>...
4 long long findSecondShortestTime (int n, int m,
  vector<vector<int>> > arr) {
5     // Write your code here
6
7 > }...
```

Autocomplete

Ln 1, Col 1

Tab: 4 Spaces

Visual Studio

Test cases Quick test Results

Test case 1**Sample Input 1**

```
4
4
1 2 3
2 4 4
1 3 2
3 4 6
```

Sample Output 1

8

Copy to custom input

Next >

Previous

1/2

58:56

Run

Submit

0/2 100

50.00

C++17 (g++ 10.3.0)

C++17 (g++ 10.3.0)

Sample Input 1

Sample Output 1

```
4
4
1 2 3
2 4 4
1 3 2
3 4 6
```

```
8
```

Explanation

Routes from station 1 to station 4 and their total times:

1 → 2 → 4: 3 + 4 = 7 seconds.

1 → 3 → 4: 2 + 6 = 8 seconds.

1 → 2 → 1 → 2 → 4: 3 + 3 + 3 + 4 = 13 seconds, and longer doubling-back routes cost even more.

The fastest time is 7. Among all total times (7, 8, 13, ...), the smallest value strictly greater than 7 is 8.

Note: Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

```
1 > #include <bits/stdc++.h> ...
4 long long findSecondShortestTime (int n, int m,
  vector<vector<int> > arr) {
5     // Write your code here
6
7 > }
```

Autocomplete

Ln 1, Col 1

Tab: 4 Spaces

Visu

Test cases Quick test Results

Test case 1

Sample Input 1

Sample Output 1

```
4
4
1 2 3
2 4 4
1 3 2
3 4 6
```

```
8
```

Copy to custom input

Previous

1/2

Next >

44:40

Run

Submit

Section: Coding Programming

50.00



C++17 (g++ 10.3.0)



Calibration Console of the Old Mill

Deep in the cellar of the Old Mill of Bramwick sits a brass calibration console with a single dial display and three worn buttons. The display always shows a whole number between 1 and n , and every calibration session begins with the display showing exactly 1.

The three buttons behave rigidly. The first increases the displayed number by 1, the second decreases it by 1, and the third multiplies it by a fixed gear factor b . The console jams if the display would ever leave the range 1 to n , so the millwright never presses a button that would make the number less than 1 or greater than n .

Every gauge in the mill has a serial mark from 2 to n , and each gauge must be calibrated once. Calibrating the gauge with mark k means starting a fresh session at 1 and pressing buttons until the display shows exactly k . The millwright always uses the minimum possible number of button presses for each gauge.

The mill's ledger requires the total wear on the console. This is the sum, over all marks k from 2 to n , of the minimum number of button presses required to reach k from 1.

Compute this total.

Function Description

Implement the function `computeCalibrationCost`.

Parameters

- n : The largest number the display can show. The gauges have marks from 2 to n .
- b : The fixed multiplication factor of the third button.

```
1 > #include <bits/stdc++.h> ...
4 long long computeCalibrationCost
  (int n, int b) {
5     // Write your code here
6
7 > }
```

Autocomplete

Ln 1, Col 1

Tab: 4 Spaces

Visual Studio

Test cases

Quick test

Results

< Previous

2/2

Next >

44:23

Run

Submit

Section: Coding Programming

50.00



Returns

- The sum of the minimum number of button presses required to reach every number from 2 to n, starting from 1 for each number.

Input Format

- The first line contains a single integer n, the display limit.
- The second line contains a single integer b, the multiplication factor.

Output Format

- Print a single integer representing the total minimum number of button presses required for all gauge marks from 2 to n.

Constraints

- $3 \leq n \leq 100000$
- $2 \leq b \leq 10$
- All values are integers.

Sample Input 1

19
4

Sample Output 1

23

C++17 (g++ 10.3.0)

```
1 > #include<bits/stdc++.h>--
4 long long computeCalibrationCost
  (int n, int b) {
5     // Write your code here
6
7 > }
```

Autocomplete

Ln 1, Col 1

Tab: 4 Spaces

Visual Studio

Test cases Quick test Results

< Previous

2/2

Next >



Section: Coding Programming

50.00



C++17 (g++ 10.3.0)



1

2

Explanation

Starting each session at 1, with buttons +1, -1, and *4 (never leaving the range 1 to 10).
Mark 2: press +1 (1 → 2). Presses = 1. Mark 3: press +1, +1 (1 → 2 → 3). Presses = 2. Mark 4: press *4 (1 → 4). Presses = 1. Mark 5: press *4, +1 (1 → 4 → 5). Presses = 2. Mark 6: press *4, +1, +1 (1 → 4 → 5 → 6). Presses = 3. Mark 7: press +1, *4, -1 (1 → 2 → 8 → 7). Presses = 3. Mark 8: press +1, *4 (1 → 2 → 8). Presses = 2. Mark 9: press +1, *4, +1 (1 → 2 → 8 → 9). Presses = 3. Mark 10: press +1, *4, +1, +1 (1 → 2 → 8 → 9 → 10). Presses = 4.
Total = 1 + 2 + 1 + 2 + 3 + 3 + 2 + 3 + 4 = 21.

Note: Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, Java 17, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, R(RScript), Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic

```
1 > #include <bits/stdc++.h>--
4 long long computeCalibrationCost
  (int n, int b) {
5     // Write your code here
6
7 > }
```

Autocomplete

Ln 1, Col 1

Tab: 4 Spaces

Visual Studio

Test cases Quick test Results

< Previous

2/2

Next >